

BCC2001

Algoritmos



Diego Bertolini
diegobertolini@gmail.com

Aula 5 – Funções

- **Aula Anterior:**

- Laços de Repetição ;

- **Aula de Hoje:**

- Escopo: O tempo de vida de uma variável ;
- Funções em Linguagem C ;
- Bibliotecas ;

Bloco de Comandos

- **Blocos de código formados por um ou mais comandos que são tratados como uma unidade.**
- **Em C, os delimitadores de bloco: { e };**
- **Utilizados normalmente em conjunto com outros comandos: if(), while(), for();**

Escopo de Variáveis

- **Algumas regras a partir do conceito:**

- As variáveis valem no bloco que são definidas;
- As variáveis definidas dentro de uma função recebem o nome de variáveis locais;
- Os parâmetros formais de uma função valem também somente dentro da função;
- Uma variável definida dentro de uma função não é acessível em outras funções, mesmo que tenham nomes idênticos.

Escopo de Variáveis

- **Trata sobre a acessibilidade das variáveis.**
 - Escopo de uma variável entende-se o bloco de código onde esta variável é válida.
- **Escopo local: visível em um local específico**
 - **Exemplo:** variáveis locais de uma função.
- **Escopo global: visível a todo o programa.**
 - **Exemplo:** variáveis globais do programa.

Escopo Global

- Variável declarada no escopo global ou simplesmente Variável Global ;
- É uma variável declarada fora de todas as funções do programa, ou seja, na área de declarações globais do programa (acima da cláusula *main*, juntamente com as bibliotecas do programa ;
- Essas variáveis existem enquanto o programa estiver executando, ou seja, o tempo de vida de uma variável global é o tempo de execução do programa ;
- Estas variáveis podem ser acessadas e alteradas em qualquer parte do programa ;

Escopo Global

```
1  #include <stdio.h>
2
3  int x = 5 ;
4
5  // Função Incrementa.
6  void incrementa(){
7      x++ ;
8  }
9
10 int main(){
11
12     printf("x = %d \n", x);
13     incrementa() ;
14     printf("x = %d \n", x);
15     incrementa() ;
16     printf("x = %d \n", x);
17     x = 9999 ;
18     printf("x = %d \n", x);
19
20     return 0 ;
21 }
```


Escopo Global

- De modo geral, evita-se o uso de variáveis globais em um programa ;
- Prejudica a manutenção do programa ;
- Torna-se mais difícil saber onde a variável é inicializada ;
- Além disso variáveis globais ocupam memória durante todo o tempo de execução do programa e não apenas quando são necessárias ;

Escopo Local

- Variáveis declaradas no escopo Local ou variáveis Locais ;
- É uma variável declarada dentro de um bloco de comandos delimitado pelo operador de chaves ({ }, escopo local).
- Essas variáveis são visíveis apenas no interior do bloco de comandos onde foram declaradas, ou seja, dentro do seu escopo.

Escopo Local

```
1  #include <stdio.h>
2
3
4  int main(){
5
6      int x = 2 ;
7      int y = 5 ;
8
9      if (x < y){
10         int varLocal = 666 ;
11         printf("Variável Local = %d \n", varLocal) ;
12     }
13
14     printf("x = %d \n", x) ;
15     printf("y = %d \n", y) ;
16     //printf("Variável Local = \n", varLocal) ;
17
18     return 0 ;
19 }
20
```

```
Variável Local = 666
x = 2
y = 5
```

Escopo Local

```
1  #include <stdio.h>
2
3
4  int main(){
5
6      int x = 2 ;
7      int y = 5 ;
8
9      if (x < y){
10         int varLocal = 666 ;
11         printf("Variável Local = %d \n", varLocal) ;
12     }
13
14     printf("x = %d \n", x) ;
15     printf("y = %d \n", y) ;
16     printf("Variável Local = \n", varLocal) ;
17
18     return 0 ;
19 }
20
```

```
/home/diegobertol...      In function 'main':
/home/diegobertol... 16    error: 'varLocal' undeclared (first use in this function)
/home/diegobertol... 16    note: each undeclared identifier is reported only once for each function it appears in
                          === Build finished: 1 errors, 0 warnings (0 minutes, 0 seconds) ===
```

Escopo Local

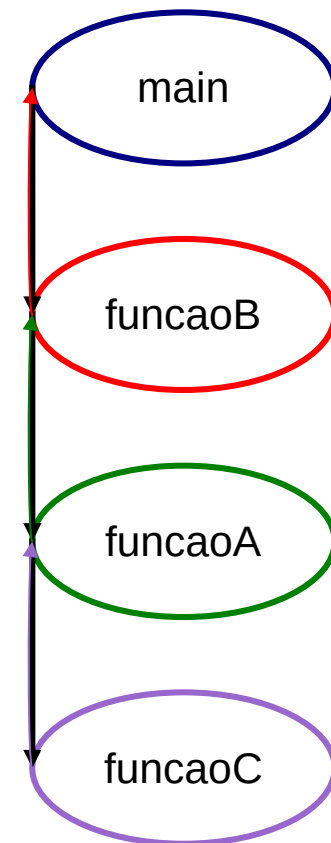
```
1  #include <stdio.h>
2
3  int x = 999; // global
4
5  int main(){
6      printf("1) x = %d \n", x) ;
7
8      int x = 666 ; // local
9      printf("2) x = %d \n", x) ;
10
11     if (1 == 1){
12         int x = 333 ; // local dentro de local
13         printf("3) x = %d \n", x) ;
14     }
15
16     x = x * 2 ;
17     printf("4) = %d \n", x) ;
18
19     return 0 ;
20 }
21
```

```
1) x = 999
2) x = 666
3) x = 333
4) = 1332
```

```

2
3// declara uma variavel global.
4int x = 1;
5
6/* Função A */
7void funcaoA(){
8    printf("funcaoA: %d\n", x);
9    x = 4;
10    funcaoC();
11}
12
13/* Função B */
14void funcaoB(){
15    int x = 3;
16    printf("funcaoB: %d\n", x);
17    funcaoA();
18}
19
20/* Função C */
21void funcaoC(){
22    printf("funcaoC: %d\n", x);
23}
24
25/* Programa principal */
26int main(){
27    funcaoB();
28    return 0;
29}
30

```



resultado
funcaoB: 3
funcaoA: 1
funcaoC: 4

Escopo Local

- Como o escopo é um assunto delicado e pode gerar confusão, evita-se o uso de variáveis com o mesmo nome ;
- Quando um bloco possuir uma variável local com o mesmo nome de uma variável global, esse bloco dará preferência à variável local. O mesmo vale para duas variáveis locais em blocos diferentes: A declaração mais próxima sempre ter maior precedência e oculta as demais variáveis com o mesmo nome.

Atividades:

- Faça um programa que calcule a média aritmética de três notas e imprima a média na tela, porém todas as variáveis devem ser globais.

Funções

- É o conceito de subprogramas ;
- Permite modularizar um programa → divisão em várias partes ;
- É um conjunto de comandos agrupados em um bloco que recebe um nome e através deste nome pode ser invocado (chamado).

Por que usar?

- Reaproveitamento de código existente.
- Evitar que um trecho de código que seja repetido várias vezes dentro de um mesmo programa.
- Permitir a alteração de um trecho de código de uma forma mais rápida. Com o uso de uma função é preciso alterar apenas dentro da função que se deseja;
- Para que os blocos do programa não fiquem grandes demais → mais difíceis de entender;
- Facilitar a leitura do programa-fonte → legibilidade
- Para separar o programa em partes (blocos) que possam ser logicamente compreendidos de forma isolada.

Funções de entrada e saída

- Exemplo que já vimos de uso de funções.

```
1 #include <stdio.h>
2 int main (void){
3
4     int i;
5     float f;
6     char c;
7
8     printf("Digite os valores: \n");
9     scanf("%d %f %c", &i, &f, &c);
10
11     printf("Valores digitados: \n");
12     printf("Inteiro: %d\n", i);
13     printf("Float: %f\n", f);
14     printf("Caracter: %c\n", c);
15
16     return 0;
17 }
```

```
rogerio@ragnote:/dado/hello_c$ ./entrada
Digite os valores:
1
23.5
t
Valores digitados:
Inteiro: 1
Float: 23.500000
Caracter: t
rogerio@ragnote:/dado/hello_c$ ./entrada
Digite os valores:
1 23.5 t
Valores digitados:
Inteiro: 1
Float: 23.500000
Caracter: t
```

Declarando uma função

```
tipo_da_funcao NomeDaFuncao (lista_de_parametros)
{
    /* corpo da função */
}
```

- A Lista de Parâmetros (lista_de_parametros) ou Lista de Argumentos, é opcional.
- O próprio main (programa principal) é uma função.

```
int main(){
    /* Corpo do programa principal */
    return 0;
}
```

Declarando uma função

```
#include <stdio.h>

int Quadrado (int a){
    return (a*a) ;
}

int main(){
    int n1, n2 ;
    printf("Entre com um número: ") ;
    scanf("%d", &n1) ;

    n2 = Quadrado(n1) ;

    printf("O quadrado de %d é = %d \n", n1, n2) ;
    return 0 ;
}
```

Parâmetros

- A utilização de parâmetros torna o uso de funções um pouco mais amplo.
- Eles determinam sobre quais dados a função trabalhará.
- A função `printf("Olá Mundo!")` recebe por parâmetro a mensagem *"Olá Mundo!"* e a imprime na saída padrão.

Parâmetros

- A definição de parâmetros é feita da mesma forma que a declaração de variáveis, entre os parênteses do cabeçalho da função.
- Caso precise declarar mais de um parâmetro, basta separá-los por vírgulas.

Parâmetros

```
tipoRetornado nome_da_função (tipo Nome1, tipo Nome2, tipo Nome3, ..., tipo NomeN){  
    corpo da função ;  
}
```

- Mesmo se não houver parâmetros na função os parênteses são necessários ;
 - ***void mensagem()***
 - ***void mensagem(void)***
- No exemplo a seguir temos a função SOMA que possui dois parâmetros, sendo o primeiro um *float* e o segundo um *int*.

Parâmetros

```
float soma(float a, int b){ // parâmetros formais separados por vírgulas.  
    float resultado;        // a declaração de variáveis é igual a de um programa.  
  
    resultado = a + b;       // Operações e expressões trabalham da mesma forma.  
  
    return resultado;        // O resultado deve ser retornado, pois o retorno esperado  
                             // é um float.  
}  
  
/* Programa principal */  
int main(){  
    float s = soma(1,2);    // chamada à função soma com os parâmetros reais.  
  
    return 0;  
}
```

Parâmetros

Colocar (void) na lista de parâmetros é diferente de não colocar nenhum parâmetro ;

O parâmetro de uma função é uma variável local da função e, portanto, só pode ser acessado dentro da função.

```
1  #include <stdio.h>
2  // Erro Comum...
3  int Quadrado (int a, b, c){
4      return (a*a) ;
5  }
6  // Modo correto!
7  int Quadrado (int a, int b, int c){
8      return (a*a) ;
9  }
10
```

Parâmetros

```
1  #include <stdio.h>
2
3  void mensagem (void){
4      printf("Teste da Função \n") ;
5  }
6
7  void mensagem2 (){
8      printf("Teste da Função \n") ;
9  }
10
11
12  int main(){
13      //mensagem();
14      mensagem2();
15
16      //mensagem(5);
17      mensagem2(5);
18
19      //mensagem(5, 'a');
20      mensagem2(5, 'a');
21
22      return 0 ;
23  }
24
```

```
Teste da Função
Teste da Função
Teste da Função
```

```
1  #include <stdio.h>
2
3  void mensagem (void){
4      printf("Teste da Função \n") ;
5  }
6
7  void mensagem2 (){
8      printf("Teste da Função \n") ;
9  }
10
11
12  int main(){
13      mensagem();
14      //mensagem2();
15
16      mensagem(5);
17      //mensagem2(5);
18
19      mensagem(5, 'a');
20      //mensagem2(5, 'a');
21
22      return 0 ;
23  }
24
```

```
/home/diegobertol... 16  error: too many arguments to function 'mensagem'
/home/diegobertol... 3    note: declared here
/home/diegobertol... 19  error: too many arguments to function 'mensagem'
/home/diegobertol... 3    note: declared here
== Build finished: 2 errors, 0 warnings (0 minutes, 0 seconds) ==
```

Corpo da Função

- Pode-se dizer que o corpo da função é sua alma ;
- É no corpo da função que se define a tarefa que ela vai realizar quando for chamada ;
- Tudo o que temos dentro de uma função `main()` pode ser feito em uma função desenvolvida pelo programador .
- De modo geral, evita-se fazer operações de leitura e escrita dentro de uma função .

Corpo da Função

- O ideal é que a função somente receba os parâmetros necessários, e devolva o resultado do calculo realizado pela função ;

```
1  #include<stdio.h>
2  #include<math.h>
3
4  float raiz (float x){
5      return (pow(x,0.5));
6  }
7
8  int main(){
9      float x, sq ;
10     printf("Digite um número: ");
11     scanf("%f", &x);
12     sq = raiz(x) ;
13
14     printf("A raiz quadrada de %f é = %f \n\n\n ", x, sq);
15     return 0 ;
16 }
17
```

Retorno da Função

- O retorno da função é a maneira como uma função devolve o resultado (se ele existir) da sua execução para quem a chamou .
- Assim, `tipo_retornado`, estabelece o tipo de valor que a função vai devolver para quem a chamar ;
- Uma função pode retornar qualquer tipo válido em Linguagem C.

```
tipo_retornado  NomeDaFuncao (lista_de_parametros)
{
    /* corpo da função */
}
```

Retorno da Função

- Uma função também pode NÃO retornar um valor. Para isso, basta colocar o tipo **void** como valor retornado.
- Para executar uma função do tipo **void**, basta colocar no código onde a função será chamada o nome da função e seus parâmetros ;

Retorno da Função

- Se a função não for do tipo void, ela deverá retornar um valor. O comando `return` é utilizado para retornar esse valor para o programa:
 - **return** expressão ;
 - A expressão da cláusula **return** tem de ser compatível com o tipo de retorno declarado para a função ;
 - Para executar uma função que tenha o comando **return**, basta atribuir a chamada da função, a uma variável compatível com o tipo do retorno.

Retorno da função (funções com Retorno)

- O valor que será retornado para o chamador da função.
- Deve respeitar o tipo de retorno declarado.

```
float soma(float a, int b){ // parâmetros formais separados por vírgulas.
    float resultado;       // a declaração de variáveis é igual a de um programa.

    resultado = a + b;      // Operações e expressões trabalham da mesma forma.

    return resultado;      // O resultado deve ser retornado, pois o retorno esperado
                           // é um float.
}

/* Programa principal */
int main(){

    float s = soma(1,2);    // chamada à função soma com os parâmetros reais.

    return 0;
}
```

A blue curved arrow originates from the function call `soma(1,2)` in the `main` function and points to the opening curly brace of the `soma` function definition, illustrating the flow of control and data return.

Retorno da função

- O valor que será retornado para o chamador da função.
- Deve respeitar o tipo de retorno declarado.

```
float soma(float a, int b){ // parâmetros formais separados por vírgulas.
    float resultado;        // a declaração de variáveis é igual a de um programa.

    resultado = a + b;       // Operações e expressões trabalham da mesma forma.

    return resultado;        // O resultado deve ser retornado, pois o retorno esperado
                             // é um float.
}

/* Programa principal */
int main(){
    float s = soma(1,2);    // chamada à função soma com os parâmetros reais.
    return 0;
}
```

The diagram illustrates the return flow of the function. A green arrow originates from the `return resultado;` statement in the `soma` function and points to the `float s = soma(1,2);` line in the `main` function. A blue arrow points from the `soma(1,2)` expression to the variable `s`, indicating the assignment of the returned value. A black arrow points from the `return` statement to the `soma(1,2)` expression, showing the return path.

Retorno da Função

- Uma função pode ter mais de uma declaração **return** (Utilizando if-else, switch).
- Quando se chega a um comando **return** a função é encerrada imediatamente ;

Parâmetros Formais

- Os parâmetros da função na sua declaração são chamados parâmetros formais.

```
float soma(float a, int b)
```

Parâmetros Reais

- Na chamada da função os parâmetros são chamados parâmetros reais ou atuais.

```
soma(1, 2)
```

Passagem de parâmetros

- Dois tipos:
 - **Por valor**
 - **Por referência**
- Os parâmetros são passados para uma função de acordo com a sua posição.
- O primeiro parâmetro real (da chamada) define o valor o primeiro parâmetro formal (na definição da função)
- O segundo parâmetro atual define o valor do segundo parâmetro formal e assim por diante.
- Os nomes dos parâmetros na chamada não tem relação com os nomes dos parâmetros na definição da função.

Passagem por valor

- Os parâmetros reais devem ser do mesmo tipo que os parâmetros formais, respectivamente.
- Se foi declarado (formalizado):

```
float soma(float a, int b)
```

A chamada à função:

```
soma(1.0, 2)
```

deve respeitar a ordem estabelecida na declaração formal da função.

O primeiro parâmetro é um *float* e o segundo é um *int*.

Passagem por valor

- As alterações feitas a um parâmetro formal NÃO modifica o parâmetro real correspondente.

Passagem por valor

```
1  #include <stdio.h>
2
3  void teste(int x, float y){
4      x = 10;
5      y = 5.3;
6      printf("Dentro da Função Teste: (x,y) = (%d, %.1f)\n", x, y);
7  }
8
9  /* Programa principal */
10 int main(){
11     int x = 2;
12     float y = 1.5;
13     printf("Main_1: (x,y) = (%d, %.1f) \n", x, y);
14     // Chama a Função
15     teste(x,y);
16     // Após a Função
17     printf("Main_2: (x,y) = (%d, %.1f)\n\n", x, y);
18
19     return 0 ;
20 }
```

```
Main_1: (x,y) = (2, 1.5)
Dentro da Função Teste: (x,y) = (10, 5.3)
Main_2: (x,y) = (2, 1.5)
```

Passagem por referência

- Os parâmetros reais devem ser endereços de memória.
- Os parâmetros formais devem ser ponteiros.
- Se foi declarado (formalizado):

```
float soma(float *a, int *b)
```

A chamada à função:

```
float x = 1.2;
```

```
int t = 5;
```

```
soma(&x, &t)
```

Passagem por referência

- A alteração de um parâmetro formal **MODIFICA** o parâmetro real correspondente.

```
1  #include <stdio.h>
2
3  void teste(int *px, float *py){
4      *px = 10;
5      *py = 5.3;
6      printf("Dentro da Função: (x,y) = (%d, %.1f) \n", *px, *py);
7  }
8  /* Programa principal */
9  int main(){
10     int x = 2;
11     float y = 1.5;
12     printf("Antes: (x,y) = (%d, %.1f) \n", x, y);
13     teste(&x, &y);
14     printf("Depois: (x,y) = (%d, %.1f) \n\n ", x, y);
15
16     return 0 ;
17 }
18
```

Antes: (x,y) = (2, 1.5)
Dentro da Função: (x,y) = (10, 5.3)
Depois: (x,y) = (10, 5.3)

Forma de Organizar

- A princípio podemos tomar com regra:
 - *“toda função deve ser declarada antes de ser usada”*
- Foi o que fizemos até agora → definir as funções antes de usa-las.
- Na definição da função está implícita a declaração.
- Pode-se preferir que a função principal (main) seja a primeira parte do programa.
- A linguagem C permite que se declare uma função, antes de defini-la.
- Esta declaração é feita através do protótipo da função, que é a assinatura da função → trecho de código que especifica o nome e os seus parâmetros.

Exemplo de prototipação de funções

- Prototipadas antes de serem usadas, sendo chamadas antes de serem definidas.

```
fprototipada.c
1#include <stdio.h>
2
3/* Declaração das Funções -> Protótipos */
4float somar(float p0p1, float p0p2);
5float subtrair(float p0p1, float p0p2);
6float multiplicar(float p0p1, float p0p2);
7float dividir(float p0p1, float p0p2);
8
9/* Programa principal */
10int main(){
11    float x = 1.5; float w = 2; float s = 0;
12
13    s = somar(x,w);
14    printf("Adição: %.2f\n", s);
15    s = subtrair(x,w);
16    printf("Subtração: %.2f\n", s);
17    s = multiplicar(x,w);
18    printf("Multiplicação: %.2f\n", s);
19    s = dividir(x,w);
20    printf("Divisão: %.2f\n", s);
21
22    return 0;
```

Exemplo de prototipação de funções

- Prototipadas antes de serem usadas, sendo chamadas antes de serem definidas.

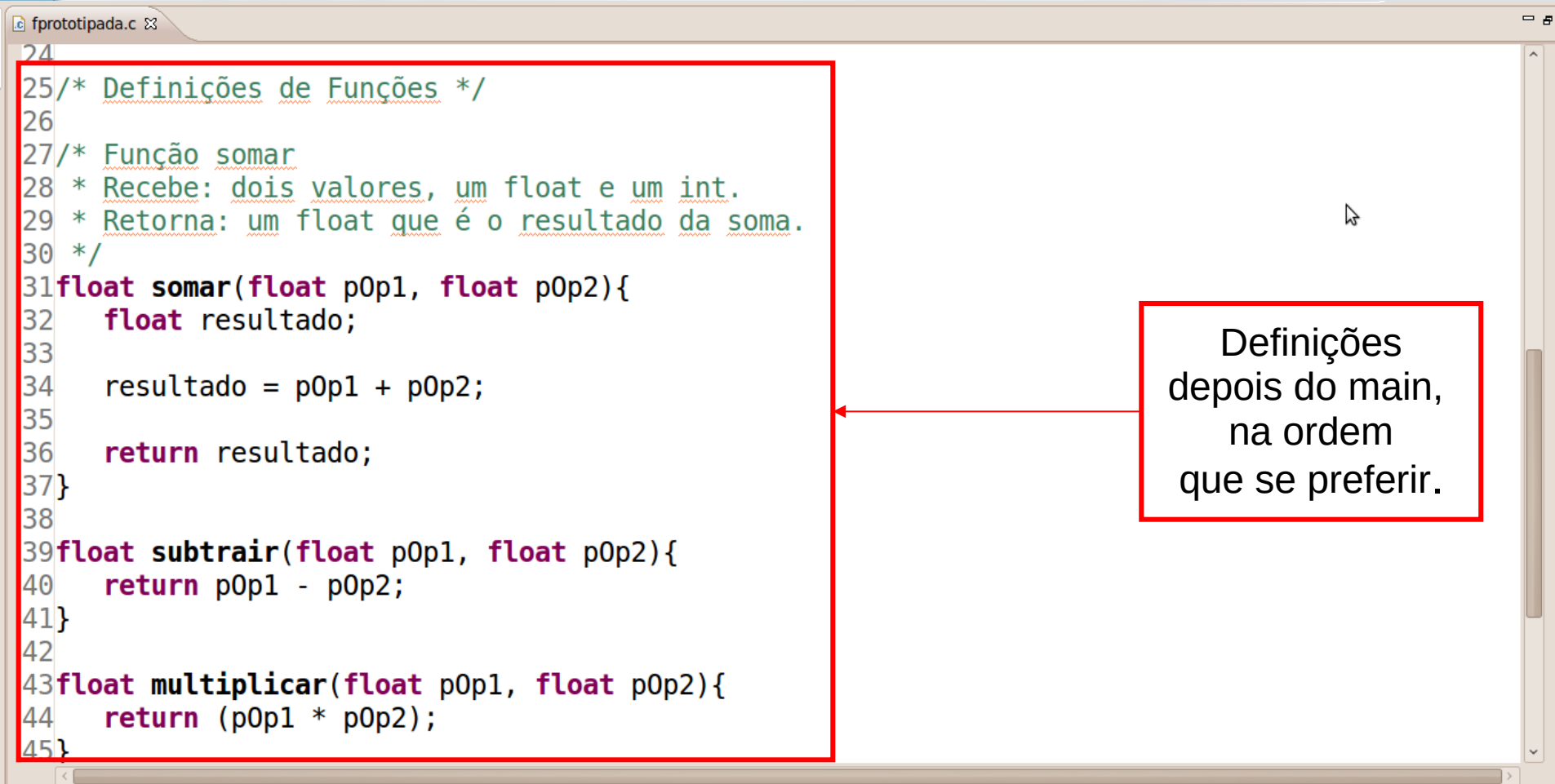
```
1#include <stdio.h>
2
3/* Declaração das Funções -> Protótipos */
4float somar(float p0p1, float p0p2);
5float subtrair(float p0p1, float p0p2);
6float multiplicar(float p0p1, float p0p2);
7float dividir(float p0p1, float p0p2);
8
9/* Programa principal */
10int main(){
11    float x = 1.5; float w = 2; float s = 0;
12
13    s = somar(x,w);
14    printf("Adição: %.2f\n", s);
15    s = subtrair(x,w);
16    printf("Subtração: %.2f\n", s);
17    s = multiplicar(x,w);
18    printf("Multiplicação: %.2f\n", s);
19    s = dividir(x,w);
20    printf("Divisão: %.2f\n", s);
21
22    return 0;
```

Declaradas

Chamadas

Exemplo de prototipação de funções

- Definições das Funções depois do main.



The image shows a code editor window titled 'fprototipada.c'. The code defines three functions: 'somar', 'subtrair', and 'multiplicar'. The 'somar' function is highlighted with a red box, and a red arrow points from a text box to it. The text box contains the text: 'Definições depois do main, na ordem que se preferir.'

```
24
25/* Definições de Funções */
26
27/* Função somar
28 * Recebe: dois valores, um float e um int.
29 * Retorna: um float que é o resultado da soma.
30 */
31float somar(float p0p1, float p0p2){
32    float resultado;
33
34    resultado = p0p1 + p0p2;
35
36    return resultado;
37}
38
39float subtrair(float p0p1, float p0p2){
40    return p0p1 - p0p2;
41}
42
43float multiplicar(float p0p1, float p0p2){
44    return (p0p1 * p0p2);
45}
```


Verificação de Parâmetros

- Os parâmetros atuais (aqueles da chamada da função) devem ser dos mesmos tipos dos parâmetros formais.
- Se isto não ocorrer, dependendo do compilador, ele pode se encarregar de converter automaticamente (cast automático).
- No caso de você declarar corretamente a função antes de usá-la, a conversão de tipos é feita como no caso de variáveis.

Verificação de Parâmetros

Na segunda chamada, a função **multiplicar** é chamada com os parâmetros reais dos tipos **int** e **float**, nesta ordem. Como na declaração da função o primeiro parâmetro é **float** e o segundo é **int**, é feita uma conversão de **int** para **float** e de **float** para **int**, respectivamente.

```
verif_param.c
1 #include <stdio.h>
2
3 /* Declaração das Funções -> Protótipos */
4 float multiplicar(float p0p1, int p0p2);
5
6 /* Programa principal */
7 int main(){
8     float x = 1.5;
9     int w = 2;
10    float s = 0;
11
12    s = multiplicar(x,w);
13    printf("Multiplicação: %.2f\n", s);
14    printf("Multiplicação: %.2f\n", multiplicar(w,x));
15
16    return 0;
17}
18
19 /* Definições de Funções */
20 float multiplicar(float p0p1, int p0p2){
21     return (p0p1 * p0p2);
22 }
```

```
Multiplicação: 3.00
Multiplicação: 2.00
```

Exercícios

- 1) Modificar o programa do cálculo da média para que utilize uma função `media(...)` no cálculo.
- 2) Ler um número e chamar uma função que retorna o dobro e o quadrado do número.
- 3) Fazer uma calculadora com as funções: soma, multiplica, divide, subtrai, potencia, raiz.

Dúvidas, Críticas ou Sugestões

diegobertolini@gmail.com